

This guide covers the prompting patterns specific to Claude Sonnet 5.5. For the model's API changes, see [What's new in Claude Sonnet 5.5](#). For techniques that apply across all current Claude models, see [Prompting best practices](#).

Existing Claude Sonnet 5 prompts should perform well without changes, and the patterns in [Prompting Claude Sonnet 5](#) remain a reasonable starting point. For the hardest long-horizon work, an Opus model is the better choice. Start with the section that matches what you observe:

- Unsure which effort level to run, or turns run longer or shorter than they did on Claude Sonnet 5: [Calibrate effort](#)
- The model stops to check in before a coding task is done, or does more than you asked: [Steer initiative and scope](#)
- Your integration runs with thinking off today: [Running without up-front thinking](#)
- JSON answers to tasks that need a few steps of working out are wrong or don't parse: [Reasoning tasks with JSON output](#)
- Long agentic turns look silent: [User-facing progress updates](#)
- The model answers from its training knowledge when a search would catch details that have changed: [Tool use in chat and knowledge work](#)
- Messages users send mid-task are ignored or treated as injected text: [Mid-turn user messages](#)
- Code changes are reported as done without a test or build run: [Verification on coding tasks](#)
- The model calls a tool with the wrong letter case or passes a parameter under a slightly different name: [Tolerant tool-call handling](#)
- Answers about dense charts or technical drawings miss detail: [Tools for complex visual inputs](#)
- Requests return `stop_reason: "refusal"`: [Safeguard refusals](#)

Calibrate effort

[Effort](#) is the main control for how much Claude Sonnet 5.5 thinks, and with it quality, latency, and cost. Its levels are recalibrated: a level doesn't produce the same amount of thinking as the same level on Claude Sonnet 5. Run a fresh sweep against your own evals rather than carrying over the setting you used on Claude Sonnet 5. Start at `high`, the default on the Claude API, unless your workload is agentic or latency-sensitive. For agentic coding and multistep tool use, start at `medium` for well-specified tasks and move to `high` for harder or longer ones. For chat and other latency-sensitive work, start at `medium` or `low`, because higher effort means a longer wait before the reply starts. Raise effort if quality needs it.

Lower effort also changes how the model finishes agentic work. At `low`, it keeps its thinking short and can skip verifying a change. See [Verification on coding tasks](#). At `low` and `medium`,

on long agentic tasks, it's more likely to stop and check in with the user before it finishes. See [Steer initiative and scope](#).

Three adjustments help:

- Set `max_tokens` with room for thinking and the reply you expect. Thinking counts toward `max_tokens` even when thinking content isn't returned to you. A limit sized for a request without thinking can cut the reply off. For agentic coding, set `max_tokens` to 128,000, the model's maximum, and [stream](#) the response.
- Reserve `xhigh` and `max` for work where you've measured a quality gain, because thinking and replies get much longer there. At those levels, `between_tools` isn't accepted, so up-front thinking can't be turned off.
- To get less thinking, lower the effort level. From `medium` up, the model thinks briefly before almost every reply, even a greeting, which adds to the time before the first visible token. Asking it in the system prompt to think less doesn't reliably reduce its thinking. At `low`, it skips thinking on most simple requests.

Changing the top-level `effort` value between requests invalidates the prompt cache. To run individual turns at a different level, use a [per-message effort change](#) (beta) instead, which keeps the cache. For example, run an interactive session at `low` and raise effort to `high` when the user submits a hard problem. Per-message effort changes need adaptive thinking. With `between_tools`, they return a 400 error, as [Running without up-front thinking](#) explains.

Steer initiative and scope

How far Claude Sonnet 5.5 goes on its own depends on the effort level and the request. At lower effort, it sometimes checks in before a coding task is done. At higher effort, or on an open-ended request, it can do more than you asked. Steer it with the effort level and with instructions in your system prompt.

Carrying work through. On agentic coding tasks at `low` and `medium` effort, the model sometimes checks in before the work is done. It might pause to confirm a plan, ask a question it could answer itself, or stop after one part of a multipart task to ask whether to continue. Try a higher effort level first. To keep the model working without changing effort, add this to your system prompt:

```
Keep working until everything the user asked for is done, and only stop to ask when you can't go on without the user or before a risky step.
```

```
When the work the user asked for is done and checked, stop and report. Don't add features, tests, files, docs or refactors that weren't asked for. If you think one would help, mention it at the end instead of doing it.
```

With this prompt, the model carries more of the work through at `low` and `medium` effort, so sessions at those levels run longer and cost more. The prompt doesn't replace your own rules

about risky or irreversible actions. Keep those rules in your system prompt.

Unrequested additions when coding. The model tends to add tests, documentation, and small supporting files that fit your repository's conventions, even when you don't ask for them. It does this at every effort level, and more at higher effort. The requested change itself stays close to what was asked. Most teams will welcome this. If you prefer changes limited to what was explicitly requested, add only the second paragraph of that prompt, which starts "When the work the user asked for is done". At `xhigh` and `max` effort, that paragraph reduces these additions and makes changes smaller overall.

Thoroughness at `xhigh` and `max` effort. At these levels the model is especially thorough. After it finishes a task, it can start its own rounds of review and verification, sometimes with subagents if your harness provides them. It can also make related fixes it noticed along the way. This takes more time and tokens, so run routine work at `high` or below, where it's rare. If you do want that extra thoroughness of these effort levels, but want to direct it to the task itself, add this to your system prompt:

```
When the work the user asked for is done and its checks pass, stop and report.
Don't start extra rounds of review or hardening on your own, and don't launch
reviewer sub-agents unless the user asked for a review. If you think a deeper
review is worth doing, say so at the end.
```

In testing on coding tasks at `max` effort, this stopped the model from launching reviewer subagents and cut session cost by about a third, with no change in quality. It makes self-started review rounds by the main agent less frequent but doesn't remove them entirely.

Open-ended requests. When a request is open-ended, for example "show me what you can do with this", the model can start building a presentation, report, or video when you only wanted ideas. If you want ideas or a plan first, say so in the request, or add this to your system prompt:

```
When the user asks for ideas, options or a plan, give them that and stop. Don't
start building or changing anything until they say to go ahead.
```

Running without up-front thinking

To run Claude Sonnet 5.5 without up-front thinking, send `thinking: {"type": "between_tools"}`. It's the lowest thinking setting on this model, and it's accepted at `high` effort or below. If your integration runs with thinking off today, switch it to `between_tools` and check these points:

- Send `between_tools` at `high` effort or below. At `xhigh` or `max` effort, a request with `between_tools` returns a 400 error. With `between_tools`, effort also can't change mid-conversation: a per-message `output_config.effort` that differs from the level in effect returns a 400 error. To vary effort per turn, use adaptive thinking. With

`between_tools`, remove any instruction that tells the model not to think. Such instructions make it more likely that the model writes internal XML tags in its visible output.

- **Read the response by block type.** With adaptive thinking, a response can begin with a `thinking` block, whose `thinking` field is empty under the default `display: "omitted"`. With `between_tools`, a response can begin with a progress-update `thinking` block. Don't assume the first content block is text.
- **Pass back the `thinking` blocks unchanged.** With `between_tools`, notes the model writes between tool calls still come back as `thinking` blocks when they run longer than a sentence or two. Each block carries a summary of the note. Pass them back unchanged with the rest of the assistant turn. A block you send back gives the model the full note it wrote, not the summary.
- **Use adaptive thinking for reasoning tasks without tools.** In a request without tools, `between_tools` means the model answers without thinking first. For tasks that need a few steps of working out, use adaptive thinking instead. See [Reasoning tasks with JSON output](#).

Reasoning tasks with JSON output

This section applies when you ask Claude Sonnet 5.5 for a JSON answer to a task that needs a few steps of working out. Examples include totaling figures from a document, applying a rule, or ranking items. On tasks like these, the model often answers without thinking first, particularly at `low` and `medium` effort. What helps depends on how you request JSON. Use [structured outputs](#) where they're available. The response text is then JSON that matches your schema, so there's nothing to parse.

With structured outputs, the response text holds only the JSON, so the model can work the problem out only in its thinking. When it skips thinking, it can be less accurate on these tasks. These changes help keep accuracy high.

Ask the model to think first. With adaptive thinking, add this line to the end of your system prompt:

```
Think the problem through before you answer.
```

With this line, the model more often thinks before it answers. At `high` effort, the line brings accuracy close to what the model reaches at `xhigh`, for a modest increase in output tokens. At `low` and `medium` effort, it raises accuracy, though not to what the model reaches at `high`, and the increase in output tokens is larger.

Or use `xhigh` effort. With adaptive thinking, `xhigh` gives the highest accuracy on these tasks even without the line. It uses more output tokens than `high`.

Use adaptive thinking rather than `between_tools`. In a request without tools, the model doesn't think before it answers under `between_tools`. The line has no effect there, and

accuracy on these tasks is lower. Use adaptive thinking for these requests, with the steps in this section. In testing, splitting the request in two, one request for the answer and one for the JSON, led to high answer accuracy and JSON compliance, but at very high cost and latency.

With structured outputs at `low` and `medium` effort, the model occasionally keeps thinking until it reaches `max_tokens`. At `high` effort and above, this almost never happens. Treat any response whose `stop_reason` is `"max_tokens"` as failed, even if its text holds valid JSON, and retry. Set `max_tokens` high enough for the thinking and the JSON, as [Calibrate effort](#) describes, but no higher than you're willing to spend on one attempt.

If you can't use structured outputs, ask for JSON in the prompt instead. The model then often works the problem out in the response text and writes the JSON at the end. The JSON usually holds the right answer, but a parser that expects the whole response to be JSON fails. Two things help:

- **Parse the last JSON value in the response.** Read only the `text` blocks, and treat a response whose `stop_reason` is `"max_tokens"` as failed. Starting at each `{` or `[`, try to parse a JSON value. When one parses, continue from the end of that value, so values nested inside it aren't counted on their own. Keep the last value found. Don't take everything from the first `{` to the last `}`. The model occasionally writes a draft before its final JSON, and that range would include both. If your answer is several JSON values in a row, such as one record per line, keep the last run of values separated only by spaces, commas, or line breaks. Check that the result has the fields you expect, and retry once if it doesn't. In testing, this made nearly every response usable without changing its accuracy.
- **Also consider `xhigh` effort with adaptive thinking.** The model then works the problem out in its thinking and nearly always returns the JSON alone. Total output tokens stay about the same as at `high`, because the working moves from the response text into the thinking.

User-facing progress updates

Between tool calls, Claude Sonnet 5.5 writes user-facing notes about what it just found and what it's doing next. Notes longer than a sentence or two come back as [progress-update thinking blocks](#). Shorter remarks stay `text`. At the default `thinking.display`, a progress-update block's text is empty, so a client that renders only `text` blocks can look silent during a long agentic turn. This matters most in chat interfaces and other products where the user follows the model's work in real time.

To show these notes, set `display: "updates"` (beta, `thinking-display-updates-2026-08-18` header). With `between_tools`, the notes come back with their summary text, so no `display` field is needed. `between_tools` takes no other field: `display`, `budget_tokens`, or `block_binding` sent with it returns a 400 error. The [migration guide](#) shows how to render the notes. Sometimes the model needs to show the user exact text partway through a long turn, such as a code snippet or a question it needs answered. For that case, give it a simple tool for

sending the user a message. Tell the model to use that tool only for such content. Declare the tool in the first request of the session, so the `tools` list doesn't change later.

Next, remove older instructions such as "hold all findings for the final response". If you then want updates at predictable points, for example a line on what the model is about to do before its first tool call and a short recap at the end, say so in the system prompt. The model follows instructions like this. Updates at set points help most in human-in-the-loop work.

If long tool-calling turns still go quiet for longer than you want, your harness can prompt an update. Have it count consecutive tool-calling steps that send the user no text or progress update. After several in a row, for example five, append a one-turn reminder after the latest tool results. Send it as a [turn-scoped system message](#) (beta), with text like this:

```
The user hasn't heard from you in a while – say in a few words what you're doing, then continue.
```

If the turn stays quiet, stop sending reminders after the second or third. Frequent harness text after tool results can make the model suspect a prompt injection, as [Mid-turn user messages](#) explains. Leave each reminder in `messages` on later requests. Because the reminder is appended rather than inserted and later deleted, the prompt cache and [preserved thinking](#) stay intact. At `high` effort, with a tool for sending the user a message available, the reminder leads the model to update the user more often and shortens its longest silent stretches, with no measurable change in task quality.

Tool use in chat and knowledge work

On chat and knowledge-work tasks, Claude Sonnet 5.5 sometimes answers from its training knowledge when a web search would catch details that have changed. Examples include what is allowed, required, or charged.

First, check your prompt for language that discourages tool use, such as "only use tools when strictly necessary" or "minimize tool calls", and remove it. Then, if your product gives the model a search tool, add this to your system prompt:

```
Use the search tool to check specifics that may have changed since your training, such as what is allowed, required or charged, even when you feel confident. For researched work such as a report or a comparison, gather current sources rather than writing from your training knowledge.
```

This matters most for research and support products, where answers depend on current details.

Mid-turn user messages

Claude Sonnet 5.5 is trained to resist indirect prompt injection, meaning malicious instructions that arrive through tool results and other content it reads during a task. Sometimes it treats a genuine user message as a possible injection. Suppose a message the user typed mid-task reaches the model as a [mid-conversation system message](#) placed directly after a tool result, or inside a `tool_result` block. The model can then tell the user that the tool result contained text posing as a message from them, and ignore the message or ask the user to confirm it.

A token countdown that your harness adds after every tool result can cause this. So can letting users send messages while the model is partway through a multistep turn, or having your harness add instructions or context after the tool results on every step. In each case, text arrives right after the tool results. With a countdown or per-step instructions, that can happen on every tool call. An occasional one-turn reminder, like the one in [User-facing progress updates](#), arrives far less often. If you see this reaction to a reminder of your own, send the reminder less often. To avoid the misread:

- Never put user text inside a `tool_result` block. The model misreads that placement most often.
- Deliver mid-turn user input as a user turn. Append the user's words as a text block in the user message that carries the `tool_result` blocks, after the last `tool_result`.
- Keep harness notices, such as reminders, in a separate mid-conversation system message after the user's words. Never put a notice and the user's words in the same block.
- In interactive sessions where users can type mid-turn, don't add your own token or budget countdown after tool results. [Task budgets](#) (beta) add a similar countdown, but they haven't been seen to cause this misread. If you see the misread while a task budget is set, try the session without one.

Verification on coding tasks

On agentic coding tasks, Claude Sonnet 5.5 generally checks its work before it reports a change as done. At `low` effort, though, it sometimes reports a change as done without running a check that exercises it. For example, it might skip the project's tests because the project's dependencies aren't installed.

If you see changes reported as complete without test or build output in the transcript, add this paragraph, or one like it, to the system prompt. At `low` effort, it makes skipped or superficial checks rare, with no measurable change in task quality and only a slightly higher cost per task:

When you change code that can be run, built, or type-checked, run a real check that exercises the change before reporting it done: the project's tests, type-checker, or build, or the changed command itself. A syntax-only check, or a check command that failed to start, does not count; if all that is missing is the project's declared dependencies, install them with its own package manager and lockfile (e.g. `npm install`, `pip install -r requirements.txt`), never via `sudo` or the system package manager, unless told not to. Only if no real check can run here, say which one you did not run and why instead of reporting the change as done.

Tolerant tool-call handling

Claude Sonnet 5.5 occasionally calls a declared tool by a name that differs only in letter case, such as `bash` for `Bash`. It can also pass a known parameter under a slightly different name. Rather than treating such a call as a fatal error, have your harness handle it in one of two ways:

- Accept the call when the match is unambiguous, even if the letter case is wrong.
- Return a `tool_result` with `is_error: true` that states the exact expected name. The model usually corrects the call on its next turn. See [Handling errors with `is\_error`](#).

Tools for complex visual inputs

For dense charts and technical drawings, give Claude Sonnet 5.5 a way to crop, zoom, or run code on the image. With such tools, the model reads these inputs markedly more accurately. On charts, the tools help at every effort level. On technical drawings, they help only from `high` effort up, and most at `xhigh` and `max`. For charts, adding tools helps more than raising effort: in testing, with tools at `high` effort, the model read charts more accurately than without tools at `max` effort, at a fraction of the cost. The [crop tool recipe](#) has a working tool definition.

Safeguard refusals

Claude Sonnet 5.5 runs safety classifiers that can decline a request. A decline arrives as a normal response with `stop_reason: "refusal"`, and `stop_details.category` names the [refusal category](#):

- `cyber`: the request could enable cyber harm, such as malware or exploit development. Finding vulnerabilities in source code is allowed. High-risk dual-use cybersecurity work isn't allowed.
- `bio`: the request could enable biological harm, such as dangerous lab methods. Everyday health and educational questions aren't affected.
- `frontier_llm`: the request could assist the development of competing AI models.
- `reasoning_extraction`: the request asks the model to reproduce its internal reasoning in the response text.

- `general_harms`: the request falls under another usage-policy area. Benign work can also trigger this category.

If the `bio` classifier blocks your organization's life sciences work, you can apply to the [Life Sciences Verification Program](#).

If you turn on [server-side fallback](#) (beta), it retries `cyber` and `frontier_llm` declines on Claude Sonnet 5. It doesn't retry `bio`, `reasoning_extraction`, or `general_harms` declines. See [Refusals, fallback, and billing](#).

If your prompts ask the model to include its reasoning in the response, remove those instructions, because they invite `reasoning_extraction` declines. With adaptive thinking, read the reasoning from [summarized thinking](#) blocks instead (`display: "summarized"`). You can still ask for a short explanation of the answer or a summary of the actions taken; see [Keep reasoning in thinking blocks](#).